

# INFORME DE EVIDENCIAS Y CORRECCIONES APLICADAS

## Proyecto

**FixYa – Plataforma de conexión entre clientes y técnicos para servicios del hogar**

## Objetivo del Informe

Presentar los resultados obtenidos durante la ejecución de pruebas funcionales sobre el backend del sistema FixYa, documentando los hallazgos detectados, las correcciones implementadas y la validación posterior realizada mediante Swagger.

---

## Metodología Aplicada

Las pruebas fueron ejecutadas manualmente utilizando Swagger como herramienta de validación de endpoints.

Se aplicaron principalmente:

- Pruebas funcionales.
- Pruebas negativas.
- Pruebas de validación de reglas de negocio.
- Pruebas de autorización y seguridad.
- Pruebas de integridad de datos.

Cada caso fue ejecutado comparando el comportamiento esperado versus el comportamiento real obtenido desde la API.

---

## Hallazgo 1

### CP-023 – Cotización duplicada para una misma solicitud y técnico

#### Problema Detectado

Durante las pruebas se detectó que un técnico podía registrar múltiples cotizaciones para una misma solicitud.

Esto provocaba:

- Duplicidad de ofertas.
- Inconsistencias en el proceso de selección.
- Posible spam de cotizaciones.

- Incumplimiento de una regla de negocio crítica.

### **Resultado Original**

El sistema respondía:

HTTP 200 OK

permitiendo crear una nueva cotización aun cuando ya existía otra asociada a la misma combinación:

Solicitud + Técnico

### **Corrección Implementada**

Se agregó una validación previa en el servicio de cotizaciones para verificar si ya existe una cotización asociada a:

- id\_solicitud
- tecnico\_usuario\_rut

Antes de realizar el registro.

### **Resultado Posterior**

El sistema ahora responde:

HTTP 400 Bad Request

Mensaje:

"Ya existe una cotización para esta solicitud y técnico"

### **Beneficio**

Se garantiza el cumplimiento de la regla de negocio que establece que un técnico solo puede emitir una cotización por solicitud.

---

## **Hallazgo 2**

### **CP-015 – Aceptación de cotización previamente anulada**

#### **Problema Detectado**

Se detectó que una cotización anulada podía volver a ser aceptada.

Flujo observado:

ENVIADA → ACEPTADA → ANULADA → ACEPTADA

Además, la cotización mantenía simultáneamente un motivo de anulación, generando una inconsistencia lógica.

### **Resultado Original**

HTTP 200 OK

La cotización volvía a estado ACEPTADA.

### **Corrección Implementada**

Se incorporó una validación de estado antes de ejecutar la aceptación.

Si el estado es:

ANULADA

el sistema bloquea la operación.

### **Resultado Posterior**

HTTP 400 Bad Request

Mensaje:

"No se puede aceptar una cotización anulada"

### **Beneficio**

Se preserva la trazabilidad del ciclo de vida de la cotización y se evita la reactivación indebida de registros anulados.

---

## **Hallazgo 3**

### **CP-103 – Técnico cotiza su propia solicitud**

#### **Problema Detectado**

Se comprobó que un técnico podía crear una solicitud y posteriormente emitir una cotización sobre ella.

Esto generaba un conflicto de interés dentro de la plataforma.

#### **Resultado Original**

HTTP 200 OK

El sistema permitía la cotización.

## **Corrección Implementada**

Se agregó una validación que compara:

- Usuario propietario de la solicitud.
- Técnico que intenta cotizar.

Cuando ambos coinciden, la operación es rechazada.

## **Resultado Posterior**

HTTP 400 Bad Request

Mensaje:

"El técnico no puede cotizar su propia solicitud"

## **Beneficio**

Se evita la manipulación del proceso de contratación y se garantiza imparcialidad en la competencia entre técnicos.

---

# **Hallazgo 4**

## **Validación de RUT Chileno**

### **Problema Detectado**

El sistema permitía registrar usuarios utilizando RUT inválidos.

No se verificaba el dígito verificador mediante el algoritmo oficial de Módulo 11.

### **Resultado Original**

El sistema procesaba el registro o generaba errores no controlados.

### **Corrección Implementada**

Se desarrolló una validación personalizada utilizando:

- Expresiones regulares (re)
- field\_validator de Pydantic
- Algoritmo Módulo 11

### **Resultado Posterior**

HTTP 422 Unprocessable Content

Mensaje:

"El RUT ingresado no es válido"

### **Beneficio**

- Se mejora la calidad de los datos.
  - Se evita registrar identidades inválidas.
  - Se fortalece la integridad del sistema.
  - Se mejora la confiabilidad de la información almacenada.
- 

## **Importancia de las Pruebas en Backend**

Aunque muchas restricciones pueden ocultarse desde el frontend, las validaciones críticas deben implementarse obligatoriamente en el backend.

Esto garantiza que:

- No puedan saltarse restricciones mediante Swagger o herramientas externas.
- Las reglas de negocio se cumplan independientemente de la interfaz utilizada.
- La base de datos mantenga información consistente.
- El sistema sea seguro frente a manipulaciones o solicitudes maliciosas.

El backend representa la última capa de protección de la aplicación.

---

## **Conclusión**

Las pruebas ejecutadas permitieron detectar vulnerabilidades funcionales y reglas de negocio incompletas dentro del sistema FixYa.

Como resultado del proceso de QA se implementaron correcciones que fortalecen:

- Seguridad.
- Integridad de datos.
- Trazabilidad.
- Control de acceso.
- Cumplimiento de reglas de negocio.

Las correcciones aplicadas fueron verificadas nuevamente mediante Swagger, obteniendo resultados exitosos y respuestas controladas por parte de la API, contribuyendo a una versión más robusta y confiable del sistema.